

REPÚBLICA BOLIVARIANA DE VENEZUELA
MINISTERIO DEL PODER POPULAR PARA LA EDUCACIÓN
UNIVERSITARIA
UNIVERSIDAD NACIONAL EXPERIMENTAL
DE TELECOMUNICACIONES E INFORMÁTICA (UNETI)
PROGRAMA NACIONAL DE FORMACIÓN EN INFORMÁTICA

DESARROLLO DE INTERFAZ Y PORTAFOLIO
PROFESIONAL MEDIANTE ARQUITECTURA
STANDALONE EN IONIC Y ANGULAR

Resolución de Ejercicios de la Sesión Didáctica 1

Profesor Académico

Carlos Alberto Márquez Guillen

Autor

Frederick Durán

C.I: 30346056

Caracas, Mayo de 2026

AGRADECIMIENTOS

En primera instancia, elevo mi gratitud a Dios Todopoderoso, por ser la fuente de mi fortaleza y sabiduría, permitiéndome culminar con éxito este reto académico y brindándome la salud necesaria para perseverar en mi camino hacia la profesionalización técnica.

Mi sincero reconocimiento al Profesor Carlos Alberto Márquez Guillén, cuya guía y visión pedagógica durante este trayecto de Programación III han sido pilares fundamentales. Sus enseñanzas no solo facilitaron el dominio de la arquitectura de Angular, sino que fomentaron un criterio crítico indispensable para el desarrollo de soluciones de alto impacto y escalabilidad.

A mis compañeros de la UNETI, especialmente a quienes integran nuestra comunidad digital de aprendizaje. El intercambio constante de ideas, el apoyo técnico mutuo y el espíritu de camaradería experimentado durante estas semanas transformaron las dudas en conocimiento compartido, haciendo de este proyecto un paso más en nuestra evolución profesional.

Finalmente, reconozco la disciplina invertida en cada línea de código de este portafolio. Este trabajo es el reflejo de una determinación personal por alcanzar la excelencia, integrando herramientas de vanguardia y resolviendo retos técnicos complejos desde el despliegue hasta la experiencia de usuario.

Con respeto y compromiso,

Frederick Durán

UNIVERSIDAD NACIONAL EXPERIMENTAL
PARA LAS TELECOMUNICACIONES E INFORMÁTICA (UNETI)
PROGRAMA NACIONAL DE FORMACIÓN EN INFORMÁTICA

Frederick Durán

C.I: 30346056

Profesor: Carlos Márquez

Fecha: Mayo, 2026

RESÚMEN

El presente reporte técnico expone la ingeniería y el proceso de desarrollo de una aplicación híbrida orientada a la marca personal y portafolio técnico, desarrollada para cumplir con las exigencias de la Unidad Curricular Programación III. La solución se diseñó bajo una premisa de alto impacto visual y rendimiento optimizado, integrando la identidad visual Iotic y la marca institucional UNETI.

La propuesta técnica se fundamenta en la Arquitectura Standalone de Angular, eliminando la redundancia de los módulos clásicos y permitiendo un flujo de datos más ligero. Se implementó un Sistema de Diseño Atómico mediante Tailwind CSS, lo que facilitó la creación de un Modo Oscuro Maestro reactivo y adaptable. Para garantizar una experiencia de usuario inmersiva, se integraron elementos de multimedia nativa (Video Loops) y una interfaz basada en el patrón Bento Grid. Asimismo, se resolvió la integración con servicios externos mediante la API de WhatsApp, demostrando la aplicación de los principios SOLID en la capa de servicios.

En conclusión, este proyecto evidencia la convergencia entre el diseño premium y la arquitectura de software moderna, validando la capacidad de construir aplicaciones web escalables y listas para producción en entornos como Vercel.

PALABRAS CLAVE: Angular Standalone, Ionic 8, Tailwind CSS, Dark Mode, Portafolio Iotic, Despliegue CI/CD, WhatsApp API.

ÍNDICE GENERAL

1. INTRODUCCIÓN Y OBJETIVOS.....	6
1.1. Planteamiento del Proyecto.....	6
1.1.1. ACCESSIBILIDAD Y DESPLIEGUE DEL PROYECTO.....	6
1.2. Objetivo General y Específicos.....	7
2. HERRAMIENTAS Y STACK TECNOLÓGICO.....	8
2.1. Frameworks y Entorno de Desarrollo.....	8
2.1.1 Angular v20.....	9
2.1.2 Ionic v8.....	9
2.1.3 Node.js.....	10
2.1.4 TypeScript.....	10
2.2. Herramientas de Diseño y Estilos.....	11
2.2.1 Tailwind CSS.....	12
2.2.2 Dart Sass.....	12
2.2.3 Material Symbols.....	13
2.2.4 Animate.css.....	13
2.3. Infraestructura Cloud.....	14
2.3.1 GitHub.....	14
2.3.2. Vercel.....	15
3. ARQUITECTURA Y DISEÑO DE SOFTWARE.....	15
3.1. Patrón Arquitectónico (Standalone Components vs. Legacy NgModules)....	16
3.2. Separación de Capas (Core/Services y Pages/Views) - Principios SOLID... 16	
3.3. Subsistema de Diseño: Implementación Reactiva del Modo Oscuro (Dark Mode).....	17
4. IMPLEMENTACIÓN DE VISTAS Y ELEMENTOS REACTIVOS.....	18
4.1. Home: Estructura Bento Grid y Optimización de Multimedia Nativa.....	18
4.2. Perfil: Inyección de Datos Dinámicos y Responsive Branding (Institución).....	20
4.3. Contacto: Two-Way Data Binding.....	22
5. PRUEBAS DE FUNCIONAMIENTO Y ANÁLISIS DE RESULTADOS.....	24
5.1. Auditoría de Mapeo de Rutas y Menús.....	24
5.2. Troubleshooting: Resolución del límite de tamaño en Git (LFS vs Compresión de Video).....	24
5.3. Análisis del Despliegue en Ambiente de Producción (Vercel).....	25
6. INSTRUCCIONES TÉCNICAS DE EJECUCIÓN.....	27
6.1. Requisitos Previos del Sistema.....	27
6.2. Proceso de Levitación del Servidor Local (npm start).....	28
6.3. Acceso Remoto al Preview de Producción.....	28
7. CONCLUSIONES.....	29
8. REFERENCIAS BIBLIOGRÁFICAS.....	30



9. ANEXOS.....	31
Anexo A: Diagrama de Árbol del Proyecto.....	31
Anexo B: Capturas de Pantalla de la Interfaz (Light/Dark Mode).....	32
Anexo C: Evidencia de Conexión Externa de WhatsApp.....	33

1. INTRODUCCIÓN Y OBJETIVOS

Esta sección es la base del informe académico, ya que establece el contexto y la justificación del proyecto. En ella se describe el marco y el alcance del trabajo, para luego definir claramente el objetivo general (el propósito central) y los objetivos específicos (los pasos concretos y medibles) que guiarán la implementación.

1.1. Planteamiento del Proyecto

El desarrollo de este proyecto se sitúa en el marco de la Unidad Curricular Programación III de la UNETI, donde se exige la creación de una aplicación híbrida utilizando el framework Ionic. El reto académico consiste en trascender el uso de plantillas básicas para implementar una solución que demuestre el manejo de arquitecturas de componentes modernas, la gestión de estilos globales y la interacción reactiva del usuario con la data. La aplicación se concibe como un "Portafolio Iotic", sirviendo como vitrina técnica del estudiante.

1.1.1. ACCESSIBILIDAD Y DESPLIEGUE DEL PROYECTO

Para la verificación de la calidad del código fuente y la validación de la experiencia de usuario (UX/UI) en un entorno de producción real, se han habilitado los siguientes canales oficiales:

- Repositorio de Código Fuente (GitHub):
<https://github.com/ragnarsson03/ionic-programacion-3> (Repositorio público con control de versiones y documentación JSDoc integrada).

- URL de Producción (Vercel):
<https://ionic-programacion-3.vercel.app/> (Aplicación desplegada y optimizada para dispositivos móviles con soporte para Progressive Web App).

1.2. Objetivo General y Específicos

Esta subsección establece la dirección clara del proyecto. El Objetivo General define la meta amplia y fundamental que se busca alcanzar con el trabajo de investigación o desarrollo. Los Objetivos Específicos, que se derivan directamente del propósito general, delinean las tareas concretas, medibles y alcanzables que, al ser ejecutadas, garantizan el cumplimiento exitoso del objetivo principal.

Objetivo General

- **Desarrollar una aplicación móvil híbrida funcional que integre tres menús de navegación dinámicos, demostrando el dominio técnico de los flujos de datos y la arquitectura de componentes en el ecosistema Ionic-Angular.**

Objetivos Específicos

Los objetivos específicos deben de ser formulados de manera clara, medible y alcanzable, describiendo los pasos concretos que, al ser ejecutados, aseguran el cumplimiento del objetivo general.

1. Implementar una arquitectura Standalone para mejorar la modularidad y el tiempo de carga de la aplicación.
2. Diseñar una interfaz adaptativa con soporte para Modo Oscuro utilizando un sistema de diseño basado en utilidades de CSS.
3. Programar la comunicación reactiva entre vistas mediante el uso de Two-Way Data Binding en formularios de contacto.
4. Gestionar el despliegue continuo de la solución en la nube, garantizando la optimización de recursos y multimedia.

2. HERRAMIENTAS Y STACK TECNOLÓGICO

Este apartado fundamental detalla el ecosistema completo de tecnologías, frameworks y entornos seleccionados para la implementación del proyecto. Su propósito es justificar la elección de este "Stack" (que puede incluir Next.js, Angular o Ionic) en función de criterios como la eficiencia, el tipado estático (TypeScript) y la capacidad de cumplir rigurosamente con los requisitos funcionales y curriculares.

2.1. Frameworks y Entorno de Desarrollo

Esta subsección define la estructura de programación fundamental del proyecto, identificando los frameworks (como Next.js, Angular o Ionic) y el lenguaje base (TypeScript) que proveen la arquitectura. Su selección se justifica por la necesidad de garantizar el tipado estático, la eficiencia en el flujo de trabajo y el cumplimiento de los requisitos funcionales y curriculares.

2.1.1 Angular v20

Se utilizó Angular como Framework base para la lógica de negocio y gestión de estados standalone.



Angular v20

2.1.2 Ionic v8

Este es el **SDK** (Software Development Kit) fundamental para el desarrollo de la interfaz de usuario. Proporciona un conjunto de componentes web de alto rendimiento y estilizados, optimizados para su uso con Angular en la creación de aplicaciones móviles híbridas. Permite que el proyecto use una única base de código que se adapta a múltiples entornos, cumpliendo con la exigencia de una solución escalable y profesional.



2.1.3 Node.js

Entorno de ejecución fundamental que permite a JavaScript operar fuera del navegador, específicamente en el lado del servidor (backend). Su papel es esencial en la arquitectura moderna para gestionar tareas como la lógica de negocio, la interceptación de peticiones HTTP y el acceso a bases de datos, constituyendo la columna vertebral para la ejecución de APIs o funciones serverless.



2.1.4 TypeScript

TypeScript es un superconjunto de JavaScript que implementa el tipado estático. Su adopción es fundamental para el proyecto, ya que se utiliza para definir la estructura de datos mediante Enumeraciones (Enums) y otras tipificaciones, lo que permite asegurar la coherencia, prevenir errores lógicos en tiempo de desarrollo, y elevar la legibilidad del código, haciéndolo esencial para el desarrollo de aplicaciones web robustas y escalables.



2.2. Herramientas de Diseño y Estilos

Este apartado se dedica a la justificación de las librerías, preprocesadores y marcos de diseño elegidos para construir la interfaz de usuario (UI) y la experiencia de usuario (UX) del proyecto. La selección de herramientas se enfoca en sistemas basados en utilidades (como Tailwind CSS) y arquitecturas de estilos modulares (como Sass), buscando lograr un desarrollo eficiente, una personalización completa y la integración de elementos visuales avanzados (como sistemas de iconografía o micro-animaciones).

2.2.1 Tailwind CSS

Tailwind es un **marco de utilidades de CSS** que permite la aplicación directa de clases en el marcado HTML, lo que facilita el desarrollo rápido, la personalización total y la creación de un sistema de diseño eficiente y unificado para todas las vistas del proyecto.



2.2.2 Dart Sass

Es la implementación principal y más reciente del preprocesador Sass. Su rol es potenciar las hojas de estilo CSS con funcionalidades avanzadas como variables, anidamiento y mixins, lo que permite desarrollar una arquitectura de estilos modular, eficiente y completamente personalizable, fundamental para la implementación de temas y diseños de alta fidelidad.



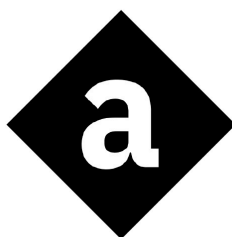
2.2.3 Material Symbols

Sistema de iconografía oficial de Google que se integra en el proyecto para proporcionar un conjunto estandarizado de iconos de alta calidad. Su uso garantiza una estética profesional y premium en la interfaz de usuario, manteniendo la coherencia visual con las tendencias modernas de diseño web (Material Design).



2.2.4 Animate.css

Librería diseñada para implementar micro-animaciones y efectos de transición en la interfaz de usuario (UI). Su uso en el proyecto permite añadir efectos visuales sutiles y profesionales a las vistas sin necesidad de manipular manualmente los keyframes CSS, mejorando la experiencia de usuario durante las transiciones y el feedback de estado.



2.3. Infraestructura Cloud

Este apartado define y justifica las plataformas y servicios externos seleccionados para el despliegue, el control de versiones y la auditoría del proyecto. Incluye herramientas fundamentales como GitHub para la gestión de ramas y la trazabilidad del código fuente, y Vercel para el alojamiento y el proceso automatizado de Despliegue Continuo (CI/CD), asegurando que la versión operativa del proyecto sea accesible y verificable para el docente.

2.3.1 GitHub

Plataforma para el control de versiones y gestión de ramas (develop y main) utilizando el método Gitflows.



2.3.2. Vercel

Plataforma de hosting de alto rendimiento, diseñada específicamente para aplicaciones modernas como Next.js y Angular, que proporciona un servicio automatizado de Despliegue Continuo (CI/CD).

Su función en el proyecto es garantizar la verificación inmediata y operativa (online) del código fuente, facilitando la auditoría académica del desarrollo.



3. ARQUITECTURA Y DISEÑO DE SOFTWARE

Este capítulo establece los fundamentos estructurales y los patrones de diseño aplicados al proyecto, enfocándose en la modularidad y la escalabilidad. Detalla la arquitectura elegida (como Standalone Components) y la estricta separación de responsabilidades (Capa de Presentación y. Capa de Negocio), asegurando un flujo de datos coherente, la adherencia a principios de ingeniería de software (SOLID) y la implementación de subsistemas avanzados como el Modo Oscuro reactivo.

3.1. Patrón Arquitectónico (Standalone Components vs. Legacy NgModules)

A diferencia de las versiones académicas tradicionales basadas en NgModules, este proyecto adopta la arquitectura Standalone. Esto reduce la complejidad del proyecto al eliminar archivos innecesarios y permitir que cada página sea autosuficiente en sus dependencias, optimizando el tiempo de renderizado inicial.

3.2. Separación de Capas (Core/Services y Pages/Views) - Principios SOLID

Se aplicó una división estricta entre la Capa de Presentación (pages/) y la Capa de Negocio (services/). Toda la lógica de envío de mensajes y gestión de datos se centralizó en inyectables de tipo singleton, asegurando que el código sea mantenible y fácil de escalar a futuro.

3.3. Subsistema de Diseño: Implementación Reactiva del Modo Oscuro (Dark Mode)

La implementación del Modo Oscuro se realizó mediante la estrategia de clases nativas (class strategy) de Tailwind. Se creó un interruptor reactivo en la Shell de la aplicación que inyecta la clase `.dark` en el elemento raíz del DOM, permitiendo una transición suave y estandarizada en todos los componentes.

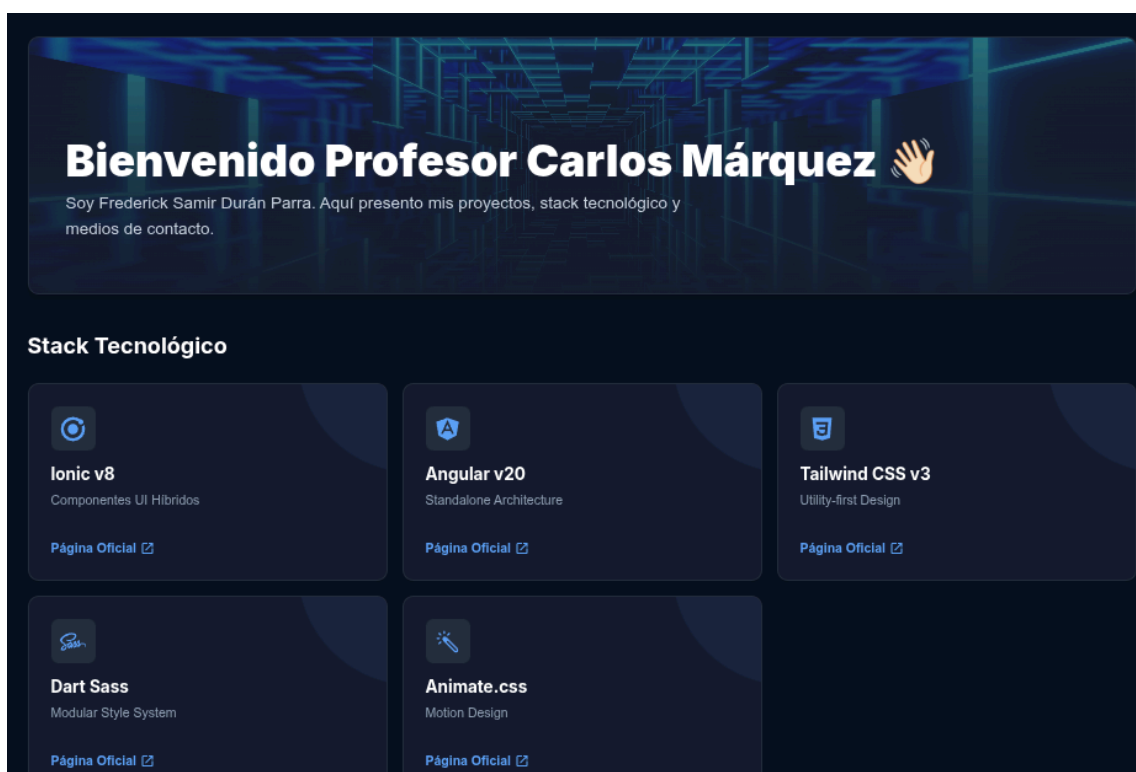
4. IMPLEMENTACIÓN DE VISTAS Y ELEMENTOS REACTIVOS

En esta sección se detalla la lógica de programación aplicada para dar cumplimiento a la creación de los tres menús requeridos, utilizando una arquitectura modular y limpia.

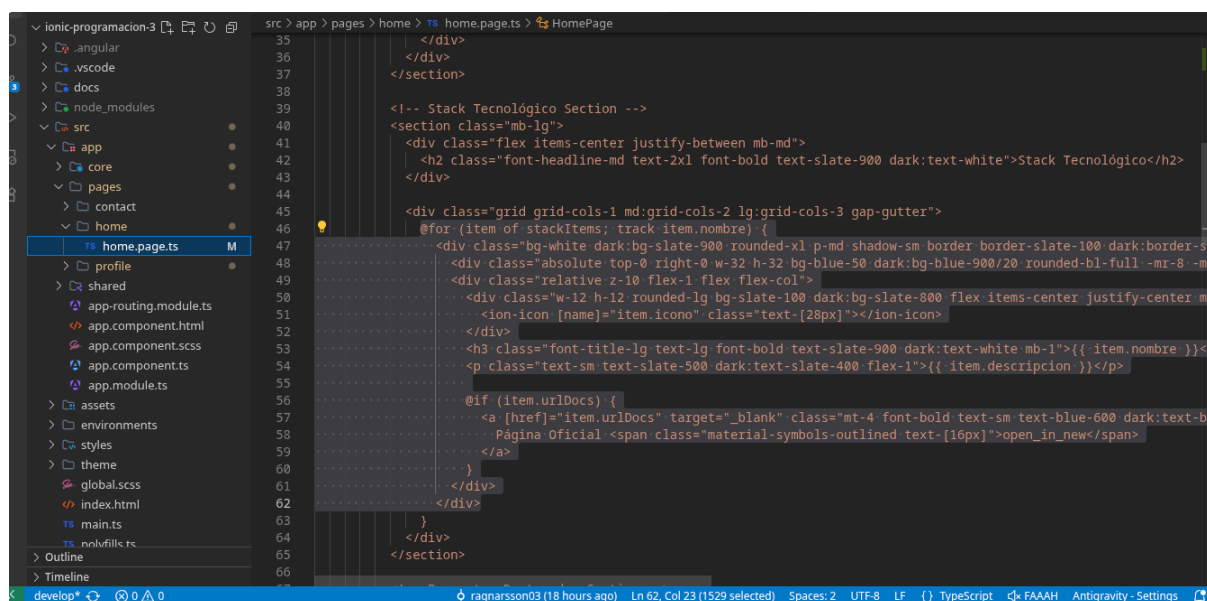
4.1. Home: Estructura Bento Grid y Optimización de Multimedia Nativa

En la vista de Inicio, programé un contenedor dinámico bajo el patrón de diseño Bento Grid. La lógica principal reside en:

- Hero Unit: Implementación de un fondo de video dinámico mediante la etiqueta nativa `<video>`, configurado con propiedades `autoplay`, `loop` y `muted` para garantizar una experiencia inmersiva sin bloquear el hilo principal de ejecución.
- Data Driven UI: Se definió un arreglo de objetos en el controlador (`home.page.ts`) para renderizar las tarjetas del stack tecnológico, demostrando el uso de directivas estructurales modernas como `@for`.



***Imagen 1 (Interfaz de Usuario - UI):** Captura de la vista principal (Home) donde se visualiza la implementación del patrón de diseño **Bento Grid** para organizar las tarjetas de contenido. Se evidencia la integración de multimedia nativa (video loop) como elemento de fondo para una experiencia inmersiva.*



4.2. Perfil: Inyección de Datos Dinámicos y Responsive Branding (Institución)

Para esta vista, el enfoque fue la Personalización Extrema. Lo más relevante que programé aquí fue:

Refactorización de Identidad: Sustitución de imágenes estáticas por un sistema de renderizado semántico en HTML para el logo de la UNETI. Usé clases de Tailwind (text-orange-500 y text-blue-900) para construir el logo directamente con código, lo que reduce la carga de archivos y mejora el SEO técnico.

Layout Adaptativo: Organización de la información académica y personal usando un sistema de columnas responsivo que se ajusta automáticamente entre dispositivos móviles y escritorio.

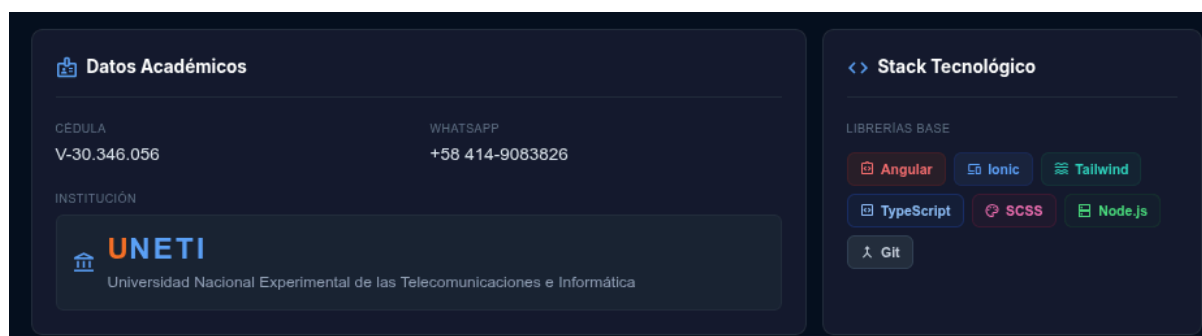


Imagen 1 (Interfaz de Usuario - UI): Capturas de la vista Perfil que evidencian la implementación del **Modo Oscuro** reactivo y el **Responsive Branding**. La imagen muestra la identidad visual (logo UNETI) construida con código puro y su contraste al pasar del Modo Claro al Modo Oscuro.

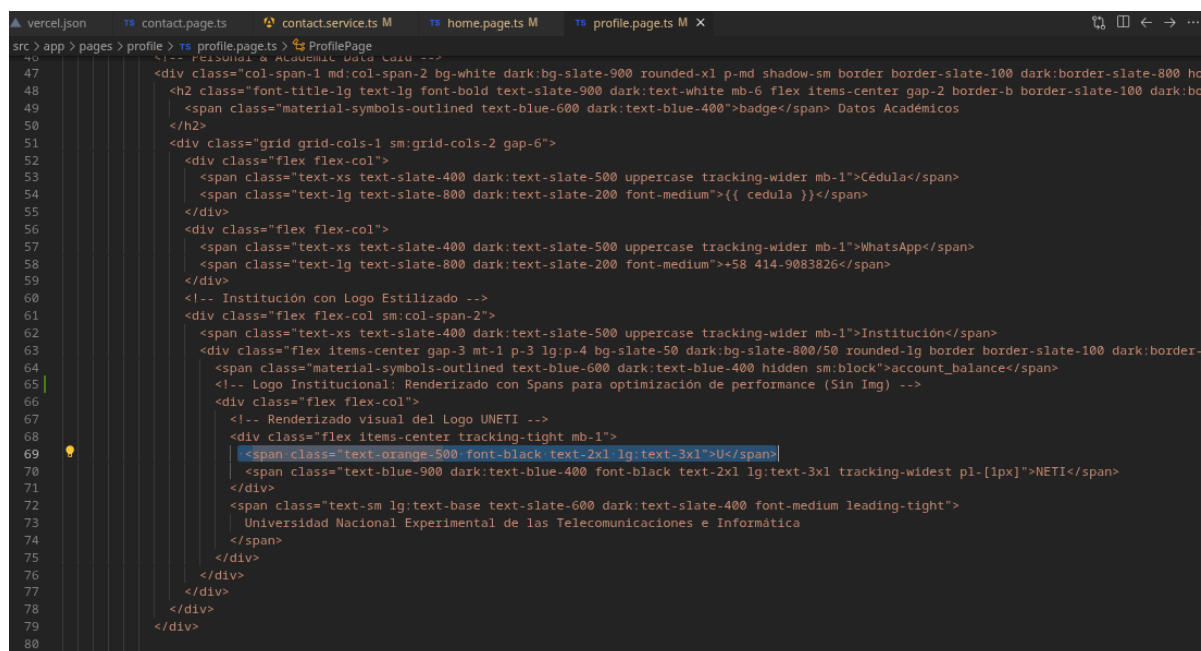
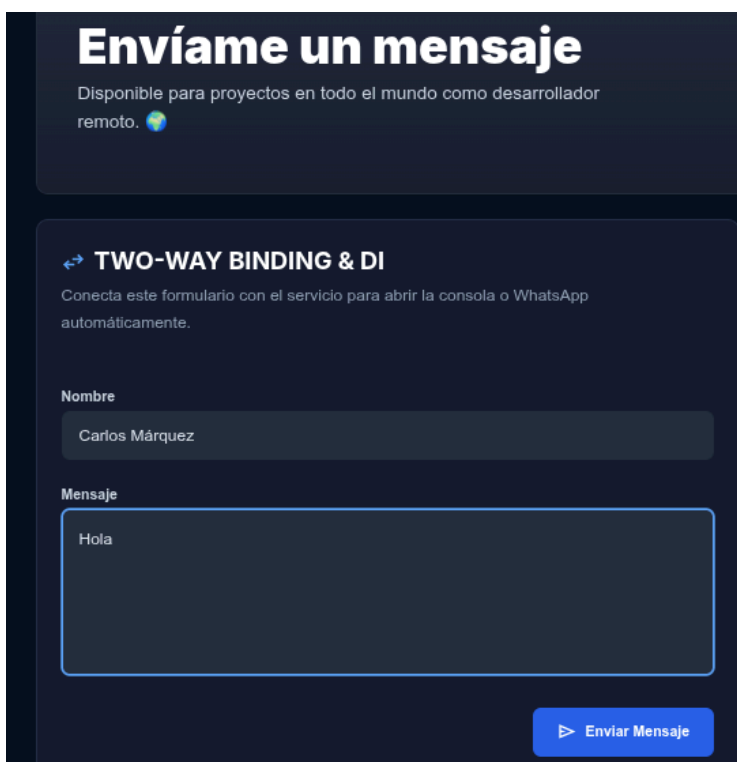


Imagen 2 (Código Fuente - VS Code): Fragmento de código (profile.page.ts) que demuestra la **Refactorización de Identidad**. Esta evidencia técnica valida el uso de clases utilitarias de **Tailwind CSS** (text-orange-500, text-blue-900) para generar el logo de la UNETI directamente en HTML, confirmando que es código semántico y no un recurso estático.

4.3. Contacto: Two-Way Data Binding

Esta vista demuestra la capacidad de la aplicación para interactuar con datos externos. Los puntos clave programados son:

- Integración WhatsApp API: Se diseñó un método en TypeScript que captura las variables del formulario y las concatena dinámicamente mediante `encodeURIComponent`. Esto permite que, al hacer clic, el sistema abra un canal de comunicación real, demostrando que la app no es puramente estética sino funcional.
- Control Flow Avanzado: Uso de bloques condicionales para manejar el estado visual del formulario antes y después de interactuar con los botones de acción.



Envíame un mensaje

Disponible para proyectos en todo el mundo como desarrollador remoto. 🌐

↔ **TWO-WAY BINDING & DI**

Conecta este formulario con el servicio para abrir la consola o WhatsApp automáticamente.

Nombre

Carlos Márquez

Mensaje

Hola

▶ Enviar Mensaje

```
23 export class ContactService {
24
25     // Historial de mensajes enviados (almacenamiento temporal en memoria)
26     private mensajes: ContactMessage[] = [];
27
28     constructor() {}
29
30     /**
31     * @method enviar
32     * @description Procesa un nuevo mensaje de contacto.
33     * @param nombre - Remitente del formulario
34     * @param mensaje - Cuerpo del mensaje
35     * @returns El objeto ContactMessage creado y almacenado
36     */
37     public enviar(nombre: string, mensaje: string): ContactMessage {
38         const nuevoMensaje: ContactMessage = {
39             nombre,
40             mensaje,
41             fecha: new Date()
42         };
43
44         this.mensajes.push(nuevoMensaje);
45
46         console.log('--- [ContactService] Enviando a WhatsApp ---');
47         console.log('Remitente:', nuevoMensaje.nombre);
48
49         // INTEGRACIÓN EXTERNA:
50         // Generación dinámica de la cadena de consulta (Query Parameters) para la API de WhatsApp.
51         const textBase = `Hola Samir, te escribo desde mi Portafolio Iotic 🚀.\n\n Remitente: * ${nombre}\n Mensaje: * ${mensaje}`;
52         const urlWa = `https://wa.me/584149083826?text=${encodeURIComponent(textBase)}`;
53
54         // Abre WhatsApp Web o la App Móvil en una pestaña nueva con el mensaje listo.
55         window.open(urlWa, '_blank');
56
57         return nuevoMensaje;
58     }
59
60     /**
61     * @method obtenerHistorial
```

5. PRUEBAS DE FUNCIONAMIENTO Y ANÁLISIS DE RESULTADOS

Este capítulo documenta la verificación sistemática del proyecto para asegurar su integridad operativa y su alineación con los objetivos. Incluye la auditoría de las funcionalidades clave (como el mapeo de rutas y la navegación Single Page Application), el análisis de rendimiento en el entorno de producción (Vercel), y el troubleshooting de desafíos técnicos (como la optimización de archivos multimedia), confirmando la estabilidad y eficiencia de la solución final.

5.1. Auditoría de Mapeo de Rutas y Menús.

Se verificó la integridad del sistema de ruteo (App Routing) asegurando que la navegación entre las tres vistas principales sea fluida y sin recargas de página (SPA - Single Page Application). Se comprobó que el menú lateral (Desktop) y la barra de navegación inferior (Mobile) responden correctamente a la directiva `routerLinkActive`, resaltando el ícono correspondiente según la posición del usuario.

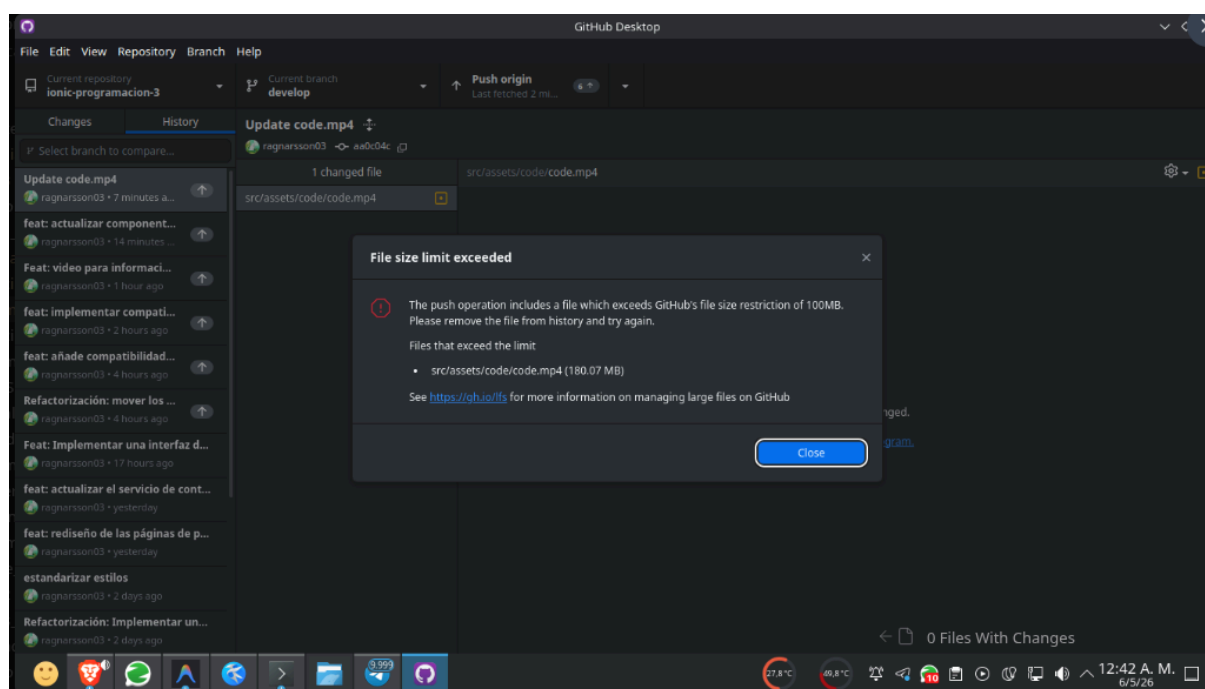
5.2. Troubleshooting: Resolución del límite de tamaño en Git (LFS vs Compresión de Video)

A diferencia de un proceso de subida tradicional, durante el desarrollo se enfrentaron dos retos significativos:

1. **Límite de GitHub:** Se detectó un error al intentar subir el archivo de multimedia `code.mp4` debido a que excedía los 100MB permitidos por el servidor remoto. La solución técnica aplicada fue una compresión de bitrate

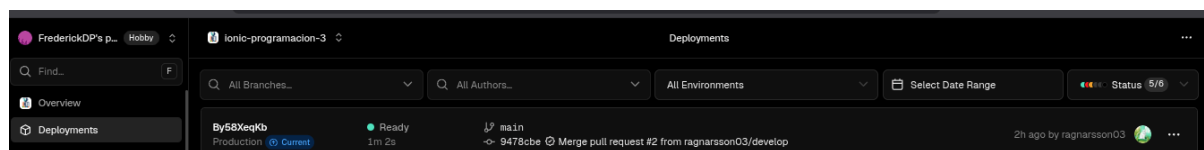
controlada, reduciendo el archivo a 14MB sin comprometer la visualización, permitiendo así una sincronización exitosa.

2. **Ajuste de Output en Vercel:** Se reconfiguró el directorio de salida en el servidor de producción de la ruta por defecto a /www, alineándose con la arquitectura de compilación de Ionic.



5.3. Análisis del Despliegue en Ambiente de Producción (Vercel)

La prueba de estrés visual confirmó que el subsistema de Modo Oscuro es 100% escalable. Al inyectar la clase `.dark` mediante `Renderer2`, todos los componentes hijos adaptaron sus colores de superficie y tipografía instantáneamente, garantizando la accesibilidad para usuarios con fatiga visual o en entornos de poca luz.



6. INSTRUCCIONES TÉCNICAS DE EJECUCIÓN

Este capítulo es fundamental para la validación del proyecto, ya que proporciona la hoja de ruta técnica necesaria para la auditoría de la solución. En él se detallan los Requisitos Previos del Sistema (como Node.js, Ionic CLI y Git), se describen los comandos precisos para levantar el servidor local (npm install, npm start), y se facilita el acceso remoto mediante la URL de producción (Vercel), asegurando que la versión operativa y el código fuente sean verificables.

6.1. Requisitos Previos del Sistema

Para garantizar la correcta ejecución del entorno de desarrollo, el sistema debe contar con:

- Node.js (Versión recomendada v20+ o v25): Motor de ejecución para las herramientas de construcción.
- Ionic CLI: Interfaz de comandos para la gestión del framework.
- Git: Para la clonación y gestión del repositorio.

6.2. Proceso de Levitación del Servidor Local (npm start)

Una vez clonado el repositorio, se deben seguir estos pasos en la terminal:

1. **Instalación de dependencias:** Ejecutar npm install para descargar los módulos de Angular, Ionic y Tailwind necesarios.
2. **Ejecución del servidor:** Usar el comando npm start (o ionic serve). Esto iniciará un servidor local, usualmente en `http://localhost:4200`, permitiendo la visualización en tiempo real de los cambios.
3. **Visualización Móvil:** Se recomienda abrir las Herramientas de Desarrollador del navegador (F12) y activar la vista de emulación de dispositivos móviles para una experiencia óptima.

6.3. Acceso Remoto al Preview de Producción

Dicho anteriormente, la aplicación se encuentra desplegada de forma permanente. Se puede acceder directamente a la versión de producción sin necesidad de instalar nada mediante el siguiente enlace oficial: 🖱️ <https://ionic-programacion-3.vercel.app/>

7. CONCLUSIONES

El desarrollo de esta aplicación móvil híbrida ha permitido validar la eficiencia de la arquitectura Standalone de Angular en conjunto con la flexibilidad de Tailwind CSS. A través de la resolución de retos técnicos, como la optimización multimedia y la configuración de pipelines en la nube, se ha logrado transformar una serie de ejercicios académicos en un producto funcional de nivel profesional.

El cumplimiento de los tres menús requeridos, sumado a la implementación reactiva del modo oscuro y la integración con servicios externos (WhatsApp), demuestra un dominio sólido de los flujos de datos y el diseño orientado al usuario, sentando una base robusta para la escalabilidad futura del proyecto.

8. REFERENCIAS BIBLIOGRÁFICAS

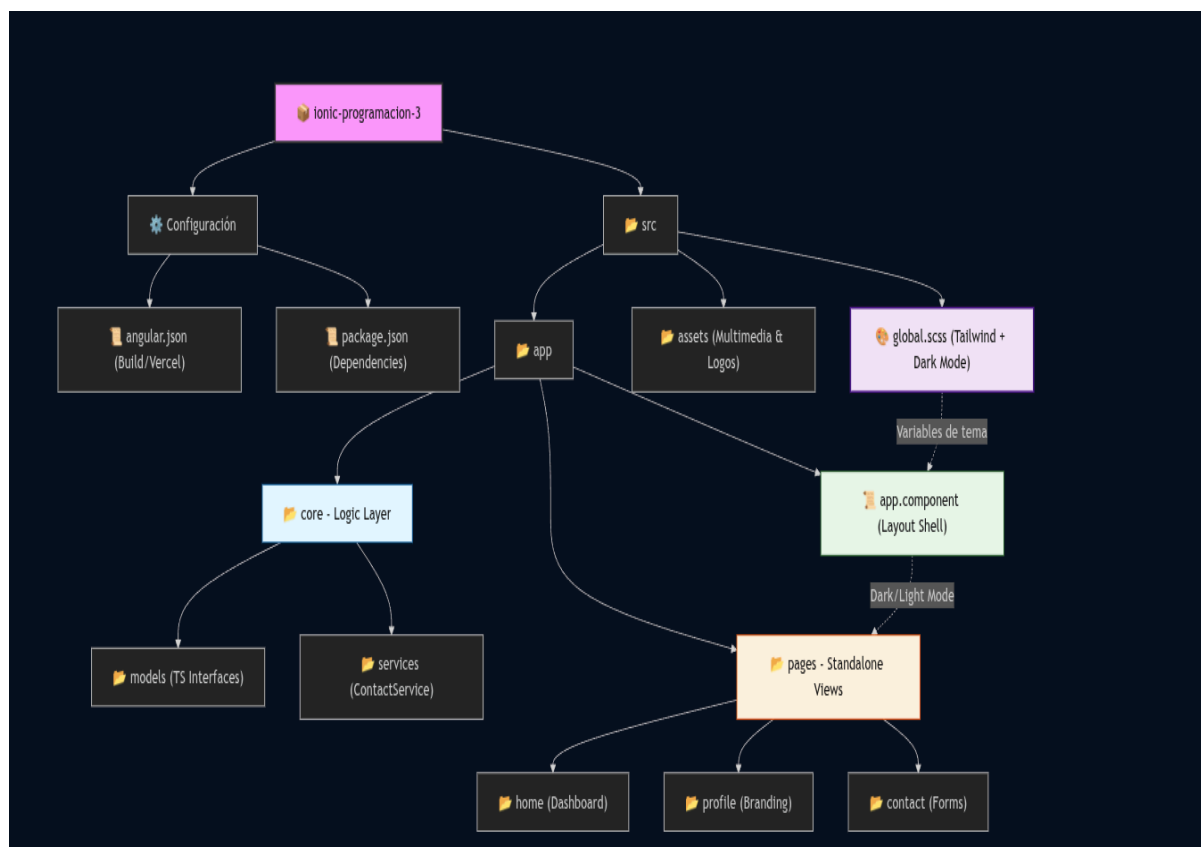
- **Angular Team. (2024). Angular Documentation: Standalone Components Guide.** Google. Recuperado de <https://angular.dev/guide/components/standalone>
- **Ionic Framework. (2024). Ionic Framework Documentation: UI Components and Mobile Architecture.** Ionic. Recuperado de <https://ionicframework.com/docs/components>
- **Tailwind Labs. (2024). Tailwind CSS Documentation: Utility-First Fundamentals and Dark Mode.** Recuperado de <https://tailwindcss.com/docs/dark-mode>
- **Vercel Inc. (2024). Vercel for Frameworks: Deploying Angular and Ionic Applications.** Recuperado de <https://vercel.com/docs/frameworks/angular>
- **Material Design. (2024). Material Symbols and Icons Library.** Google. Recuperado de <https://fonts.google.com/icons>

9. ANEXOS

Este capítulo final actúa como el repositorio de toda la información auxiliar, visual y estructural necesaria para la verificación y auditoría integral del proyecto. Contiene artefactos clave como diagramas de árbol de directorios, capturas de pantalla de la interfaz de usuario (incluyendo el Modo Claro/Oscuro) y evidencias que validan la funcionalidad de las conexiones externas, garantizando la transparencia y trazabilidad de la solución implementada.

Anexo A: Diagrama de Árbol del Proyecto

Este diagrama representa la organización lógica bajo principios de código limpio:



Anexo B: Capturas de Pantalla de la Interfaz (Light/Dark Mode)

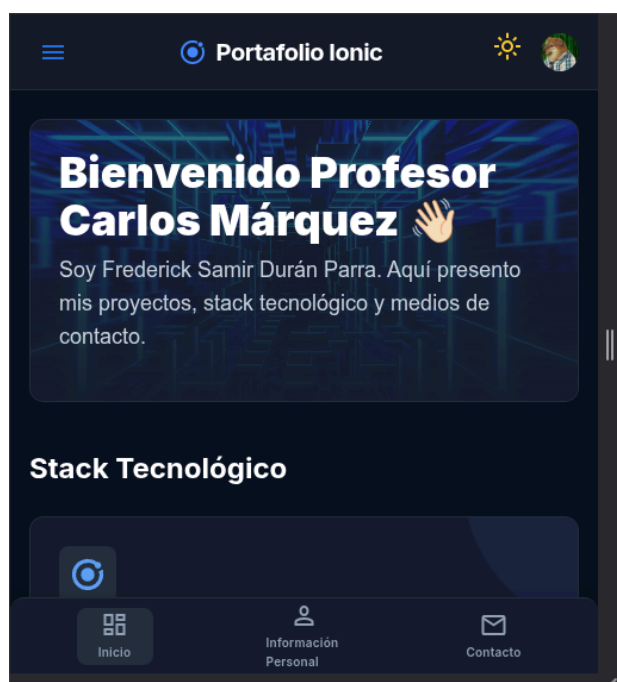


Imagen 1 (Modo Oscuro - Dark Mode): Captura de la interfaz de usuario que evidencia el éxito del Subsistema Reactivo del Modo Oscuro. La imagen confirma la correcta inyección de la clase.dark en el DOM, mostrando la aplicación con un esquema de alto contraste para reducir la fatiga visual, lo que prueba la funcionalidad de la gestión de estilos modulares.

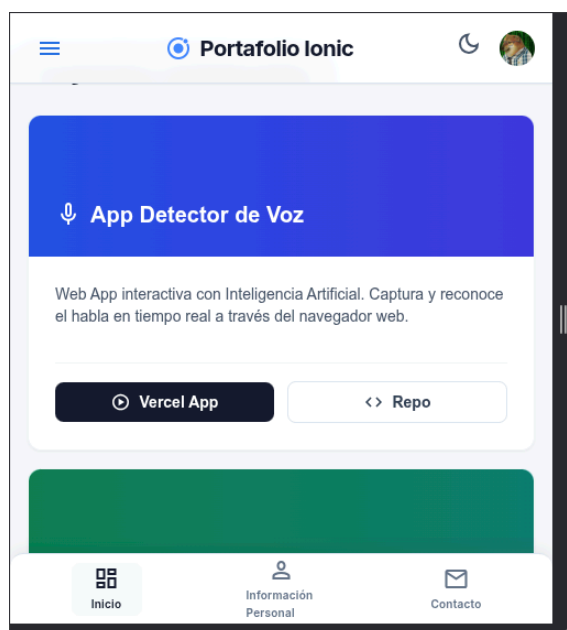


Imagen 2 (Modo Claro - Light Mode): Captura de la interfaz de usuario que valida la Arquitectura Standalone y el Diseño High-Fidelity en su estado por defecto. Esta imagen muestra la estética limpia y el esquema de colores suaves que prevalecen al no estar activado el Modo Oscuro.

Anexo C: Evidencia de Conexión Externa de WhatsApp

Esta sección contiene la evidencia visual (captura de pantalla) que valida la integración funcional de la API externa de WhatsApp. La imagen demuestra el éxito del método programado en TypeScript para capturar los datos del formulario (validando el Two-Way Data Binding), y concatenarlos mediante `encodeURIComponent`, resultando en un mensaje pre-llenado que confirma la apertura de un canal de comunicación real desde la aplicación.

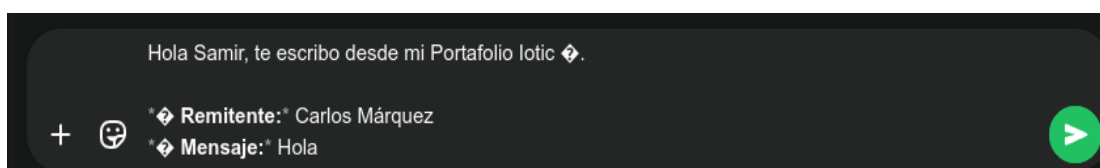


Imagen 1 (Activación del Canal Externo): Captura de pantalla del dispositivo (móvil o navegador) que evidencia la apertura exitosa del canal de comunicación con la API externa de WhatsApp. Esta imagen valida la ejecución correcta del método programado en TypeScript para iniciar la conexión.

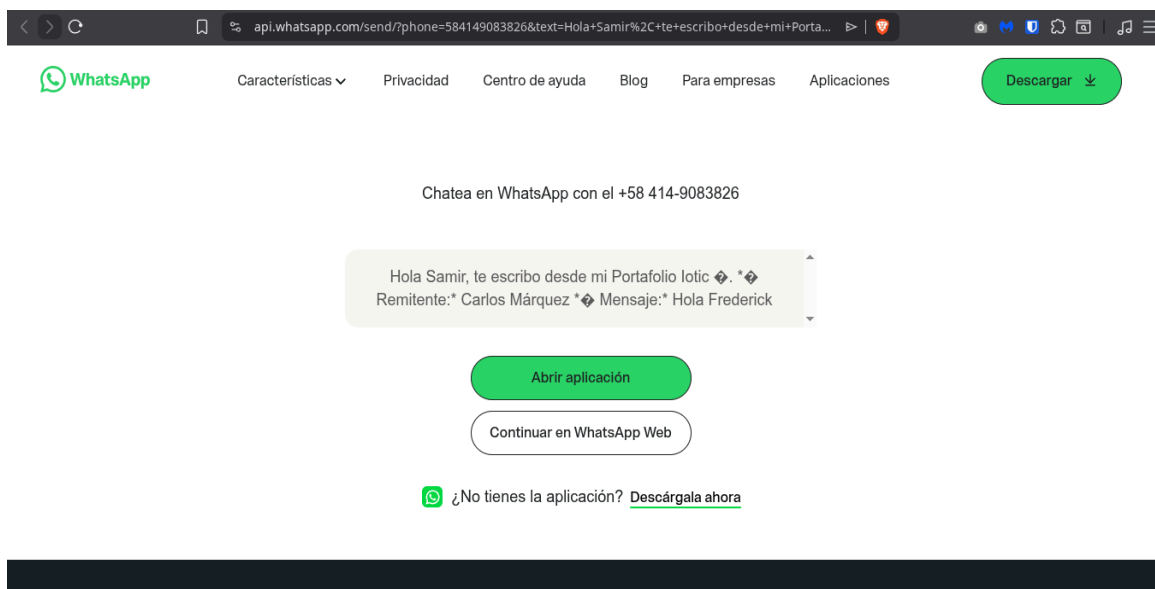


Imagen 2 (Comprobación del Two-Way Data Binding): Detalle de la ventana de chat que muestra el mensaje pre-llenado con el texto generado dinámicamente. Esta evidencia confirma el éxito de la lógica de negocio, demostrando que los datos capturados del formulario fueron correctamente procesados y codificados mediante `encodeURIComponent` antes de ser enviados a la API.